

Huawei Health Kit Cloud Integration Guide (Mobile: Android and iOS)

How a mobile app gets permission from the user and reads workout data from Huawei Health Kit Cloud. Two ways to get permission. One REST API to read the data.

PREPARED ON

September 17, 2026

FONT

HarmonyOS Sans (Regular, Medium, Bold)

SOURCES

developer.huawei.com,
github.com/Explore-In-HMOS-Wearable/hmos-user-guides

Table of Contents

1. Overview	3
1.1 The Full Flow in 4 Steps	3
1.2 What Is in This Guide	3
2. Key Ideas	4
2.1 Authentication and Authorization	4
2.2 What Does "Login-Free" Mean?	4
2.3 Method 1 and Method 2 Side by Side	5
2.4 Which Method Should You Choose?	5
3. Getting Permission	6
3.1 Required Scopes	6
3.2 Method 1: Web OAuth (Hybrid Container)	6
3.3 Method 2: Login-Free Authorization SDK	8
4. Reading Workout Records (Step 3)	13
4.1 API Basics	13
4.2 Query Parameters	13
4.3 Request Headers	14
4.4 Example Request	14
4.5 Example Answer	14
4.6 More Pages (Cursor)	15
5. Common Errors and Fixes	16
5.1 Going Live	16
6. Sources	17

1. Overview

This guide is for mobile apps on Android and iOS. It shows how your app can read a user's workout data from **Huawei Health Kit Cloud**. Huawei calls this feature **Cloud-side Data Openness**.

The data moves like this. The user's watch or band sends data to the Huawei Health app on the phone. The Huawei Health app sends the data to the Huawei cloud. Your app then reads the data from the cloud with a **REST API**. Your app does not talk to the watch.

1.1 The Full Flow in 4 Steps

Every integration uses the same 4 steps:

Step	What happens	Who does it
1	Get an authorization code . The user signs in with a HUAWEI ID and says yes to your app.	Mobile app (Method 1 or Method 2)
2	Change the code into an access token and a refresh token .	Your server
3	Call the REST API with the access token and read the workout records.	Your server or app
4	The access token works for 1 hour. After that, use the refresh token to get a new access token.	Your server

Note Method 1 and Method 2 are different **only in Step 1**. Steps 2, 3 and 4 are the same for both methods.

1.2 What Is in This Guide

- **Section 2** explains the key words: authentication, authorization and login-free. It also compares the two methods.
- **Section 3** shows how to get permission (Steps 1, 2 and 4) with Method 1 and with Method 2.
- **Section 4** shows the REST API that reads workout records (Step 3).
- **Section 5** lists common errors and how to fix them.

Note It can take a few minutes for data to go from the watch to the cloud. If the user does not open the Huawei Health app, the new data may not be in the cloud yet.

2. Key Ideas

2.1 Authentication and Authorization

These two words are often mixed up. They mean different things:

Word	Question it answers	In Huawei Health Kit
Authentication	Who is this user?	The user proves who they are with their HUAWEI ID.
Authorization	What can this app read?	The user gives your app permission for some data, for example workout records.

Huawei's page for this topic has the title "Authentication". But the page really shows the **OAuth 2.0 authorization code flow**. In this flow, both things happen together. First the user signs in (authentication). Then the user gives permission (authorization). At the end, your app gets one **authorization code**.

Note So Method 1 and Method 2 are **not** one for authentication and one for authorization. Both methods do both jobs. Both methods give you the same result: an authorization code.

2.2 What Does "Login-Free" Mean?

In **Method 1**, your app opens a Huawei web page. The user must type their HUAWEI ID and password on this page. Then the user sees the permission screen.

In **Method 2**, the SDK first checks the phone. Maybe the user is already signed in with a HUAWEI ID on this phone (in the HUAWEI ID service on a Huawei phone, or in the Huawei Health app). In that case, the user does **not** need to sign in again. The user only sees the permission screen and taps Allow. This is why Huawei calls it **login-free**.

If the user is not signed in on the phone, the SDK still shows a HUAWEI ID sign-in screen first. So login-free is the best case, not a promise for every user.

Note Login-free does **not** mean "no permission". The user must always say yes. It only means "no sign-in page, when the user is already signed in". The user can also choose to share only some of the data.

2.3 Method 1 and Method 2 Side by Side

	Method 1: Web OAuth	Method 2: Login-Free SDK
Also called	Hybrid container	Login-Free Authorization
Which step it covers	Step 1 (plus Steps 2 and 4 on your server)	Step 1 only
SDK in the app	Not needed. Only HTTPS calls.	Needed. Android: com.huawei.hms:health. iOS: AuthLoginSDK.framework.
Sign-in page	Always. The user types HUAWEI ID and password.	Not needed when the user is already signed in on the phone.
Where the user says yes	Huawei web page in a browser, Custom Tab or WebView	HUAWEI ID screen, Huawei Health app screen, or a Huawei HTML5 page. It depends on the phone (see 3.3).
How the code comes back	Browser goes to your HTTPS redirect_uri	Your app's deep link (Scheme URL)
What you register in AppGallery Connect	Your own HTTPS callback URL	Huawei's fixed callback URL, a Scheme URL, and the login-free permission
Steps 2, 3 and 4	Same	Same

2.4 Which Method Should You Choose?

Both methods are official and fully supported. Both use the cloud. The customer chooses the method that fits the app.

- **Choose Method 1** if you do not want to add a Huawei SDK to the app, or if you also need the same flow on a web site. You must host and register your own HTTPS callback page.
- **Choose Method 2** if you want an easier experience for users who already use a HUAWEI ID or the Huawei Health app on their phone. You must add the SDK, but you do not host any web page.

3. Getting Permission

3.1 Required Scopes

A scope is one type of data that your app asks for. For both methods, you must apply for these scopes in AppGallery Connect under **Activity records**. Huawei must approve them.

Scope	What it gives you	Console path
healthkit/activityrecord.read	The workout summary: start and end time, type, duration, calories and more.	Activity records > Activity record
healthkit/activity.read	The detail points of the workout: heart rate, steps, distance, speed and more.	Activity records > Activity
healthkit/location.read	The route (GPS). Only if you need the route.	Activity records > Location

The full scope starts with `https://www.huawei.com/healthkit/`, for example `https://www.huawei.com/healthkit/activityrecord.read`. You must also always add `openid`.

Note Each scope also has a write version (`.write`). If you only read data, the `.read` scopes are enough.

3.2 Method 1: Web OAuth (Hybrid Container)

In this method you do not add any Huawei SDK. Your app opens a Huawei web page inside a browser, a Custom Tab or a WebView. In this guide we call this a **hybrid container**, because it is a web page inside a native app.

Step 1: Get the Authorization Code

Open this address in the container:

```
GET https://oauth-login.cloud.huawei.com/oauth2/v3/authorize
    ?response_type=code
    &client_id={CLIENT_ID}
    &redirect_uri={REDIRECT_URI}
    &scope=openid
        +https://www.huawei.com/healthkit/activityrecord.read
        +https://www.huawei.com/healthkit/activity.read
    &state={RANDOM_STRING}
    &access_type=offline
    &display=touch
```

Parameter	Required	Meaning
client_id	Yes	Your App ID from AppGallery Connect.
response_type	Yes	Always "code".
redirect_uri	Yes	Your HTTPS callback URL. It must be the same as the callback URL you registered. It must be URL-encoded.
scope	Yes	The scopes, with spaces between them. It must include "openid".
state	No	A random string against CSRF attacks. Huawei sends it back to you with the code.
display	Yes	"touch" for phones, "page" for computers.
access_type	No	Use "offline". Then Step 2 also gives you a refresh token.

The user signs in and taps Allow. Then Huawei sends the browser to your `redirect_uri` with code and state. Your app watches the container and reads the code from this address:

```
HTTP/1.1 302 Found
Location: https://www.example.com/?code=DQB6e3x9...&state={RANDOM_STRING}
```

If the user says no, the address has `error=access_denied` instead.

Step 2: Get the Access Token

The code works for **5 minutes** and only **one time**. Send it to your server. Your server sends this request. Keep the `client_secret` on the server, never in the app.

```
POST /oauth2/v3/token HTTP/1.1
Host: oauth-login.cloud.huawei.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code={AUTH_CODE}
&client_id={CLIENT_ID}
&client_secret={CLIENT_SECRET}
&redirect_uri={REDIRECT_URI}
```

Example answer:

```

HTTP/1.1 200 OK
Content-Type: application/json

{
  "access_token": "CgB6e3x9...",
  "expires_in": 3600,
  "id_token": "eyJrawQiOi...",
  "refresh_token": "CgB6e3x9...",
  "scope": "https://www.huawei.com/healthkit/activityrecord.read
           https://www.huawei.com/healthkit/activity.read",
  "token_type": "Bearer"
}

```

The `id_token` tells you who the user is (authentication). The `access_token` lets you read the data (authorization).

Step 4: Get a New Access Token

The access token ends after **3600 seconds (1 hour)**. Use the refresh token to get a new one. The user does not see anything. The refresh token works for **180 days**.

```

POST /oauth2/v3/token HTTP/1.1
Host: oauth-login.cloud.huawei.com
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token={REFRESH_TOKEN}
&client_id={CLIENT_ID}
&client_secret={CLIENT_SECRET}

```

The refresh token stops working at once in these cases:

- The user changes the password, or the account is frozen, changed or deleted.
- The user signs out on the phone or in the browser.
- The device is removed from the account.

Then Huawei returns error 1203 (sub-code 11205 or 31204). Mark the token as expired and ask the user for permission again (Step 1).

3.3 Method 2: Login-Free Authorization SDK

In this method you add a small Huawei SDK to the app. The SDK does Step 1 for you. It chooses the best screen for this phone. When the user taps Allow, the SDK sends the **authorization code** back to your app with a deep link (your Scheme URL). You do not host any web page. Huawei hosts the callback page.

Two Solutions Inside Method 2

Huawei gives two solutions inside the Login-Free SDK. Both use the same input parameters and both return a code in the same way. Only the screen is different.

	Solution 1	Solution 2
Android call	<code>requestLoginFreeAuthorizationIntent</code>	<code>requestAuthorizationWithPrivacyIntent</code>
iOS call	<code>authLoginAction</code>	<code>authLoginActionWithSportHealthApp</code>
Where the user says yes	It depends on the phone (see table below)	Always in the Huawei Health app
Huawei Health app needed	No	Yes. Android 16.1.4.310 or later, iOS 16.1.4.306 or later.

Solution 1: What the User Sees

Scenario	Phone	Where the user says yes
1	Huawei phone	HUAWEI ID SDK screen
2	Not a Huawei phone. Huawei Health app is installed, and the user has turned on login-free authorization for your app.	Huawei Health app screen
3	Not a Huawei phone. Huawei Health app is not installed, or login-free authorization is not turned on for your app.	Huawei HTML5 (web) page

Note On **iOS**, only scenarios 2 and 3 are possible. In scenario 3 a web page opens. So Method 2 does not always avoid a web page. But Huawei hosts this page, not you.

Solution 2: What the User Sees

Scenario	Situation	What happens
1	Huawei Health app is installed and login-free authorization is turned on for your app.	The user says yes in the Huawei Health app. The app also checks if cloud sync is on. If not, it asks the user to turn it on. Then it sends the code to your app.
2	Huawei Health app is not installed.	A browser opens the download page for the Huawei Health app. After the install, the user comes back and your app must start authorization again.

Note Solution 2 is useful because it also asks the user to turn on cloud sync. Without cloud sync, the REST API in Section 4 returns no data.

AppGallery Connect Setup

Android and iOS use the same setup. You do it one time for each app.

Where in the console	What to enter
Account service details > Callback URL	Huawei's fixed URL. Enter it exactly like this. It is not your URL: <code>https://h5hosting.dbankcdn.com/cch5/healthkit/oauth-h5/oauth-callback.html</code>
Health Service Kit app details > Others	Choose your regions and apply for the login-free authorization permission.
Health Service Kit app details > Scheme URL	Your app's deep link, for example <code>myapp://oauth/callback</code> . Huawei then gives you a Scheme Secret . This is the signature parameter in the SDK call.

The SDK call needs 5 parameters:

Parameter	Meaning	Where to find it
scopes	The scopes, must include openid	Section 3.1
state	A random string, sent back with the code	You create it
clientId	Your App ID	App services > Service information > App ID
deepLinkUrl	Your Scheme URL	App services > Scheme Information > Scheme URL
signature	Your Scheme Secret	App services > Scheme Information > Scheme Secret

Android Setup

Add the SDK. The Maven repository is `https://developer.huawei.com/repo/`.

```
dependencies {
    implementation 'com.huawei.hms:health:6.11.0.300'
}
```

Add an intent-filter for your Scheme URL to an Activity in `AndroidManifest.xml`. Then start authorization:

```
val controller = HuaweiHiHealthLite.getLoginFreeAuthController(activity)

// Solution 1:
val intent = controller.requestLoginFreeAuthorizationIntent(
    scopes, state, appId, deepLinkUrl, signature)
// Solution 2 (same parameters):
// val intent = controller.requestAuthorizationWithPrivacyIntent(
//     scopes, state, appId, deepLinkUrl, signature)

activity.startActivityForResult(intent, REQUEST_AUTH_LOGIN_FREE)
```

The result comes to your deep link Activity. Read the code like this:

```
val data = intent.data ?: return
val code = data.getQueryParameter("code") // not empty = success
val state = data.getQueryParameter("state")
val error = data.getQueryParameter("errorCode") // set when it fails
```

If the Scheme URL is wrong, the deep link cannot open. Then the error comes to `onActivityResult` in `DEEPLINK_RESULT` and `DEEPLINK_MSG`.

iOS Setup

The iOS SDK is **not** on CocoaPods or Swift Package Manager. Add it by hand:

- Download the Health Service Kit iOS SDK and unzip it.
- Copy `AuthLoginSDK.framework` into the project. Set it to **Embed & Sign**. Add it to **Link Binary With Libraries**.
- Add your Scheme URL under TARGETS > Info > URL Types.
- In `Info.plist`, add `huaweihealthwearengine` to `LSApplicationQueriesSchemes`. Keep it in the first 50 entries.
- You need iOS 13 or later and Huawei Health 15.0.8.305 or later.

```
// Call every time the app starts, on the main thread:
[[HWAAuthLoginApi sharedInstance] registerApp:appId
                                scope:scopes
                                redirectUri:deepLinkUrl
                                signature:signature
                                state:state];

// Solution 1:
[[HWAAuthLoginApi sharedInstance] authLoginAction];
// Solution 2:
[[HWAAuthLoginApi sharedInstance] authLoginActionWithSportHealthApp];

// The result comes to SceneDelegate (or AppDelegate):
- (void)scene:(UIScene *)scene
  openURLContexts:(NSSet<UIOpenURLContext *> *)URLContexts {
    // Read code and state from context.URL.
    // No code in the URL means it failed. Check the result code.
}
```

After You Get the Code

The SDK gives you **only the code**. It does not give you a token. Send the code to your server and do **Step 2** and **Step 4** from Method 1. The token address is the same.

Note Please test this. Huawei's login-free pages do not show the token request. In OAuth 2.0, the `redirect_uri` in Step 2 must be the same as in Step 1. For Method 2, this is most likely Huawei's fixed callback URL from the table above. Test it before you go live.

4. Reading Workout Records (Step 3)

This REST API returns a user's workout records for a time range. Use the access token from Section 3. It does not matter which method you used. Workouts from any Huawei watch or band, and workouts the user entered in the Huawei Health app, come from this same API.

4.1 API Basics

Field	Value
Method	HTTPS GET
URL	https://health-api.cloud.huawei.com/healthkit/v2/activityRecords
Content-Type	application/json (request and answer)

4.2 Query Parameters

Parameter	Type	Required	Meaning
startTime	String	Yes	Start time in milliseconds (13 digits). It must be before endTime.
endTime	String	Yes	End time in milliseconds (13 digits). The range can be 31 days or less .
activityType	List<Integer>	No	Workout type numbers to filter by. Empty means all types.
detailDataType	List<String>	No	Name of the detail data type you also want.
sourceType	List<Integer>	No	Where the data comes from: 0 unknown, 1 workout, 2 entered by hand, 3 course, 4 connected device, 5 found automatically. If empty, you get all records except the ones entered by hand.

Note Workouts from a watch or band usually have sourceType 1 (workout) or 4 (connected device).

4.3 Request Headers

Header	Required	Meaning
Content-Type	Yes	application/json; charset=UTF-8
Authorization	Yes	"Bearer " + access token. Note the space after Bearer.
x-client-id	No	Your App ID. Recommended.
x-version	No	Your app version.
x-caller-trace-id	No	An ID to trace the request.

4.4 Example Request

```
GET https://health-api.cloud.huawei.com/healthkit/v2/activityRecords
    ?startTime=1623018038263
    &endTime=1623078058263
    &activityType=97
Content-Type: application/json
Authorization: Bearer ***
x-client-id: ***
```

4.5 Example Answer

The answer header has x-health-app-privacy. 1 means the user gave permission in the Huawei Health app. 2 means the user did not. 3 means the user does not use Huawei Health, so no permission is needed there.

```

HTTP/1.1 200 OK
Content-Type: application/json;charset=utf-8
x-health-app-privacy: 1

{
  "activityRecord": [
    {
      "id": "1623018048263",
      "name": "test",
      "startTime": 1623018048263,
      "endTime": 1623078048263,
      "activityType": 97,
      "activeTime": 13921000,
      "timeZone": "+0800",
      "activitySummary": {
        "dataSummary": [
          {
            "dataTypeName": "com.huawei.continuous.distance.total",
            "value": [{"fieldName": "distance", "floatValue": 1786.2}]
          }
        ]
      },
      "appInfo": {"appName": "app9", "appVersion": "1", "clientId": "101278501"}
    }
  ],
  "deletedActivityRecord": [],
  "hasMoreData": false
}

```

4.6 More Pages (Cursor)

If the answer has `hasMoreData: true`, there is more data. Send the cursor value to get the next page:

```

GET https://health-api.cloud.huawei.com/healthkit/v2/activityRecords
?cursor=***

# Newest records first:
GET https://health-api.cloud.huawei.com/healthkit/v2/activityRecords
?order=endTimeDesc
&cursor=***

```

You can also use the cursor to get only the changes since your last sync. Deleted records are in the `deletedActivityRecord` list.

5. Common Errors and Fixes

Error	Reason	Fix
403 Insufficient Permission	The token does not have a needed scope.	Ask the user for permission again with the right scopes.
403 Site Cross	The user's data is in another region.	Check the Location header in the answer.
Empty data	The data is not in the cloud yet, or cloud sync is off.	Ask the user to open Huawei Health and sync. Solution 2 of Method 2 also asks the user to turn on cloud sync.
400 Bad Request	A parameter has the wrong type or format.	Check the parameters against Section 4.2.
403 Query Out of Range	The data is older than the permission date.	Old data needs a separate approval from Huawei.
Invalid redirect_uri	Method 1: the redirect_uri is not HTTPS, or it is not exactly the registered callback URL.	Register the exact HTTPS URL. Or use Method 2.
1203 (11205 / 31204)	The refresh token does not work anymore.	Mark it as expired. Ask the user for permission again.
No code in the deep link	Method 2: app ID, package name or signing certificate is not the same as in AppGallery Connect, or scopes are not approved.	Check these values. See the Result Codes page for the exact error.
Deep link does not open	Method 2: the Scheme URL is wrong, or it is not the same as in the manifest or Info.plist.	Fix the Scheme URL. On Android, read the error in onActivityResult.
Huawei Health screen fails	Method 2: the user has not accepted the Huawei Health privacy statement, or the app is old.	Ask the user to open Huawei Health once and update it.

5.1 Going Live

In test mode, only **100 users** can use your app. To remove this limit, apply for verification in AppGallery Connect. The scopes you choose in the console must match the use cases you describe in the form. The review usually takes **7 to 15 working days**. After that, the app release review usually takes 1 to 2 working days.

6. Sources

This guide uses these official pages:

- Huawei Developers: Authentication (Cloud-side Data Openness, OAuth 2.0 flow)
- Huawei Developers: Login-Free Authorization, Overview (Solution 1 and Solution 2)
- Huawei Developers: Login-Free Authorization for Android Apps (Getting Started, Preparations, Developing the Feature)
- Huawei Developers: Login-Free Authorization for iOS Apps (Getting Started, Preparations, Developing the Feature, FAQs)
- Huawei Developers: Querying Created Exercise Records (REST API reference)
- Huawei Developers: Exercise Record Data Overview (OAuth Scope Description)
- Huawei Developers: Integrating the HMS Core SDK
- GitHub: Explore-In-HMOS-Wearable/hmos-user-guides, health-service-kit/README.md

This guide is for information and development help. Huawei can change API behavior, scope names and limits. Please check the official pages on developer.huawei.com before you go live.